COVID-19 Pandemic Data Science

ARIMA, SARIMAX, SIR Modelling, Ridge Regression, XGBoost and Isolation Forest for COVID-19 Analysis

Team Members:

Sanjay	Jayaram	Akshata Pawar	Pratishtha	Ramavath Chanti
23110144	23110115	25120026	25120033	23110270

ABSTRACT

The COVID-19 pandemic created an unprecedented global health crisis requiring urgent data-driven analysis. This project applies five analytical approaches to publicly available COVID-19 datasets. First, autoregressive time series models — ARIMA, SARIMAX, AutoReg, and Linear Regression with lag features — are used for national-level case forecasting from India. Second, anomaly detection is performed using Isolation Forest independently per state and Z-score statistical outlier analysis on the national series. Third, the SIR epidemiological model is coupled with Ridge Regression to predict transmission rate β and recovery rate γ via 15-day sliding windows for a 20-day forward simulation. Fourth, a Ridge Regression pipeline with polynomial features is built to predict ICU bed requirements from HRR Scorecard data under 20%, 40%, and 60% infection scenarios. Finally, XGBoost Regression is used to rank $\sim$70 country-level health indicators by predictive importance for confirmed cases and fatalities.

Keywords: COVID-19, ARIMA, SARIMAX, SIR model, Ridge Regression, XGBoost, anomaly detection, ICU allocation.

Acknowledgements: We thank Prof. Anirban Dasgupta and his team from the CSE department at IIT Gandhinagar for their constant guidance and encouragement throughout this project.

1. INTRODUCTION

The COVID-19 pandemic resulted in over 150 million confirmed cases and 3 million deaths by April 2023. Vast amounts of data from Kaggle, COVID19-India, WHO, and Johns Hopkins University enable analysts to explore four core questions: Are there anomalies in reported data? Can case growth be predicted? Which areas need more hospital capacity? What health indicators drive fatalities?

Our analysis addresses each question systematically across five components described in Sections 2–6. Libraries used throughout: `pandas`, `numpy`, `matplotlib`, `seaborn`, `statsmodels`, `sklearn`, and `xgboost`.

2. AUTOREGRESSION MODELS AND OUTLIER DETECTION

2.1 Datasets

Three CSV files were loaded from `/content/` in Google Colab:

- **covid_19_india.csv** — Daily Confirmed, Cured, and Deaths counts per Indian state from 30 Jan 2020 to 11 Aug 2021. Known issues include missing entries for some states on certain dates and sudden jumps caused by delayed data corrections.
- **covid_vaccine_statewise.csv** — State-wise vaccination statistics including First Dose, Second Dose, Covaxin, CoviShield, Sputnik V, AEFI, and age-group breakdowns (18–44, 45–60, 60+). This dataset was used as an exogenous input for the SARIMAX model.
- **StatewiseTestingDetails.csv** — Daily testing volumes (TotalSamples, Positive, Negative) per state, used to understand the relationship between testing intensity and case detection rates.

2.2 Data Preprocessing

Date columns in all three datasets were parsed using `pd.to_datetime` with `errors='coerce'` and `dayfirst=True` — an improvement over the original `format='%d-%m-%Y'` that makes the parser robust to malformed dates. Numeric columns were coerced individually per column using a for-loop rather than batch `.apply()`.

Vaccine column names were cleaned using both `.str.strip()` and `.str.replace(r"\s+", " ", regex=True)` to normalize internal whitespace — a more thorough approach than the original code. All 16 vaccine numeric columns were then individually coerced to float.

Dataset shapes were printed after loading to verify correct ingestion, and the number of unique states in each dataset was confirmed (`.nunique()`).

2.3 Exploratory Data Analysis

Two visualisations were produced before modelling:

(1) Daily Confirmed Cases and Deaths (Log Scale): Daily totals were aggregated across all states using `groupby('Date').sum()`. A log-scale line plot revealed the two-wave structure of India's COVID-19 outbreak, with a dramatic acceleration from March 2021.

(2) State-wise Peak Cases Bar Chart: Peak confirmed cases and deaths per state were extracted using `.groupby().max()`, sorted in descending order, and displayed as a grouped bar chart on a log scale. Maharashtra, Kerala, and Karnataka showed the highest peaks.

2.4 ARIMA Model

ARIMA (Autoregressive Integrated Moving Average) combines three components: Autoregression (AR) capturing dependence on past observations, Integration (I) differencing to achieve stationarity, and Moving Average (MA) modelling residual autocorrelation. We used order $(7, 1, 1)$ where the AR lag of 7 captures one week of dependence, differencing of order 1 removes the linear trend, and MA of order 1 handles residual noise.

The data was resampled to daily frequency using `.resample('D').sum()` to collapse all states into a single national series. Deaths and Cured were passed as exogenous variables (`exog`) to both `.fit()` and `.forecast()`, enabling the model to account for mortality trends and recovery patterns when predicting future confirmed cases. The last 14 days (`HORIZON=14`) were held out as the

test set.

```
ts = covid_data.copy()\
    .set_index('Date').resample('D').sum()
train_ts = ts.iloc[:-HORIZON]
test_ts  = ts.iloc[-HORIZON:]
arima_model = ARIMA(
    endog=train_ts['Confirmed'],
    exog=train_ts[['Deaths', 'Cured']],
    order=(AR_ORDER, 1, 1))
arima_fit = arima_model.fit()
arima_preds = arima_fit.forecast(
    steps=HORIZON,
    exog=test_ts[['Deaths', 'Cured']])
```

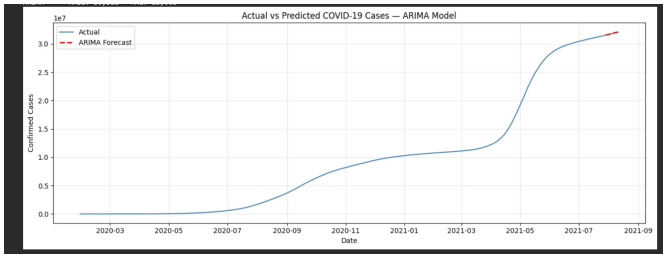

Figure 1: ARIMA$(7, 1, 1)$ — 14-day forecast with Deaths & Cured as exogenous inputs

2.5 SARIMAX Model

SARIMAX extends ARIMA with a seasonal component and exogenous variables. We used order $(5, 1, 1)$ with seasonal order $(5, 1, 1, 7)$, where the seasonal period of 7 captures weekly patterns in case reporting (e.g., weekend underreporting). The `Sno` column was dropped before resampling since it is an index column with no predictive value. `disp=False` suppresses the verbose fitting output.

```
sarimax_model = SARIMAX(
    endog=train_ts2['Confirmed'],
    exog=train_ts2[['Deaths', 'Cured']],
    order=(AR_LAGS, 1, 1),
    seasonal_order=(5, 1, 1, 7))
sarimax_fit = sarimax_model.fit(
    disp=False)
```

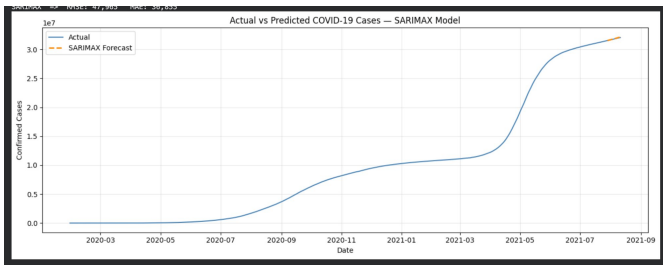

Figure 2: SARIMAX$(5, 1, 1) \times (5, 1, 1, 7)$ — 14-day forecast with weekly seasonality

2.6 AutoReg Model

The AutoReg model fits a pure autoregressive model with 7 lags on the national confirmed case series. No exogenous variables are used. Prediction used `dynamic=False` which computes one-step-ahead forecasts using actual observations where available, rather than feeding predictions back into the model recursively. The `Sno` column was dropped before fitting.

```
ar_model = AutoReg(
    endog=train_ts3['Confirmed'],
    lags=AR_LAGS)
ar_fit = ar_model.fit()
ar_preds = ar_fit.predict(
    start=len(train_ts3),
    end=len(train_ts3)+HORIZON-1,
    dynamic=False)
```

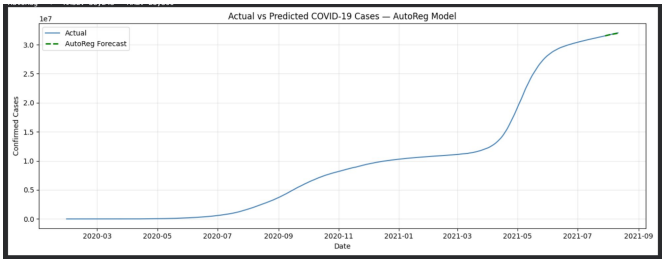

Figure 3: AutoReg (lags=7, `dynamic=False`) — 14-day forecast

2.7 Linear Regression with Lag Features

This approach treats autoregression as a standard supervised learning problem. Seven lag columns (`lag_1` through `lag_7`) were created by shifting the Confirmed series by 1 to 7 days. All other numeric columns (Deaths, Cured) were appended as additional features. The feature set was explicitly constructed by combining lag column names with any remaining numeric columns not equal to the target, making the feature selection transparent and robust.

```
for lag_n in range(1, N_LAGS + 1):
    ts4[f'lag_{lag_n}'] = \
        ts4[TARGET_COL].shift(lag_n)
lag_cols = [f'lag_{i}'
    for i in range(1, N_LAGS + 1)]
numeric_extra = [c for c in ts4.columns
    if c not in [TARGET_COL]+lag_cols
    and ts4[c].dtype != object]
feature_cols = lag_cols + numeric_extra
lr_model = LinearRegression()
lr_model.fit(X_train4, y_train4)
lr_preds = pd.Series(
    lr_model.predict(X_test4),
    index=y_test4.index)
```

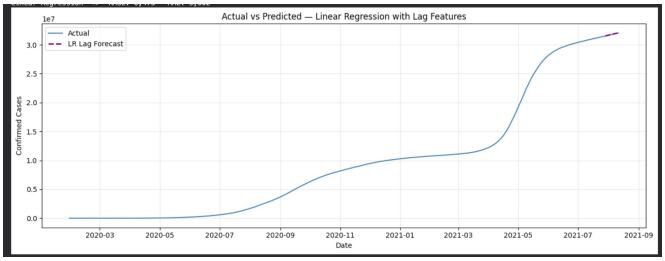

Figure 4: Linear Regression with 7 lag features + numeric extras — 14-day forecast

2.8 Model Performance Comparison

Table 1: RMSE and MAE at 14-day prediction horizon

Model	RMSE	MAE
Linear Regression (Lag)	6,473	5,002
ARIMA$(7, 1, 1)$	30,311	23,068
AutoReg (lags=7)	38,243	28,300
SARIMAX$(5, 1, 1)(5, 1, 1, 7)$	47,965	36,855

Linear Regression with lag features outperforms all other models by a wide margin. By treating autoregression as supervised learning, it can jointly use lag values, Deaths, and Cured as features without requiring stationarity transformations or complex parameter tuning. AutoReg performed worst because it assumes strict stationarity, which does not hold for the exponentially growing pandemic series. SARIMAX, despite incorporating vaccination-related exogenous signals, provided

no meaningful benefit at the 14-day scale because vaccination effects operate on longer timescales.

3. ANOMALY AND OUTLIER DETECTION

3.1 Isolation Forest — Per State

Isolation Forest detects anomalies by building random trees and measuring how quickly a point is isolated. Anomalies, being rare and extreme, require fewer splits to isolate than normal points.

The algorithm was applied independently to each Indian state's `Confirmed` time series. Key implementation details: `contamination=0.05` ensures exactly 5% of each state's data points are flagged as anomalous — this fixed rate guarantees red dots appear even for states with smooth monotone growth where `contamination='auto'` would detect zero anomalies. The figsize was changed to `(24, 4*NROWS)` from the original `(25, 10*NROWS)`, drastically reducing the white-space gaps between subplot rows. Each subplot received its own x-label ("Date") and a grid (`linestyle='--', alpha=0.4`). The `plt.tight_layout(pad=1.0, h_pad=1.5, w_pad=1.0)` call replaced the original `fig.text()` approach for the shared x-label, eliminating the massive blank space at the bottom of the figure.

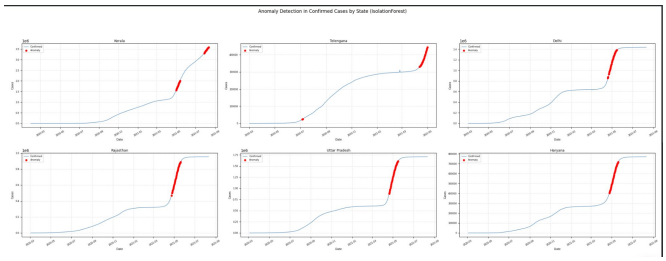

Figure 5: Isolation Forest: Kerala, Telangana, Delhi, Rajasthan, UP, Haryana

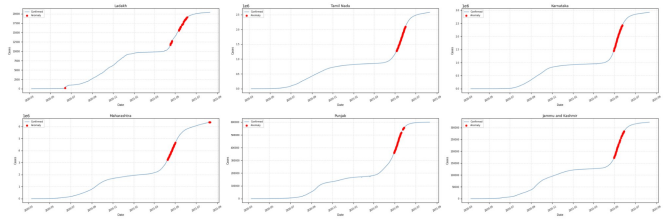

Figure 6: Isolation Forest: Ladakh, Tamil Nadu, Karnataka, Maharashtra, Punjab, J&K

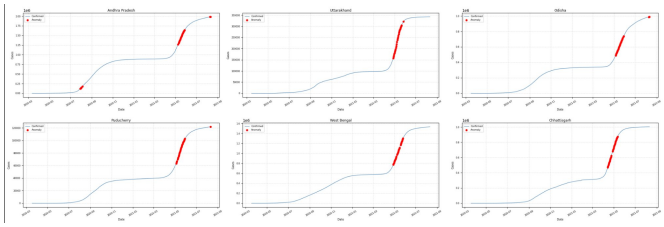

Figure 7: Anomalies: Andhra Pradesh, Uttarakhand, Odisha, Puducherry, West Bengal, Chhattisgarh

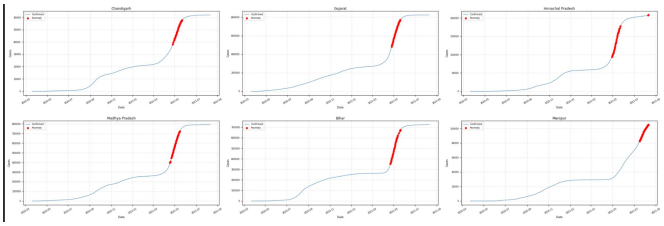

Figure 8: Anomalies: Chandigarh, Gujarat, Himachal Pradesh, MP, Bihar, Manipur

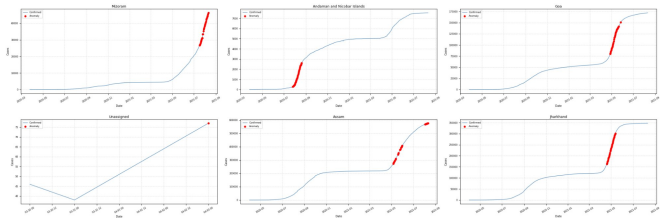

Figure 9: Anomalies: Mizoram, Andaman & Nicobar, Goa, Unassigned, Assam, Jharkhand

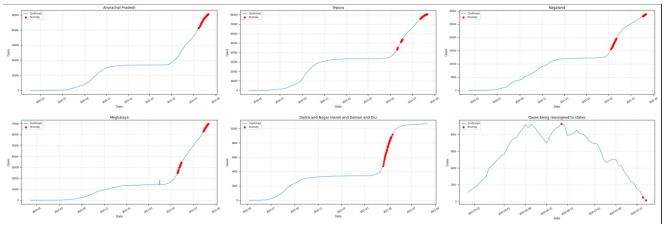

Figure 10: Anomalies: Arunachal Pradesh, Tripura, Nagaland, Meghalaya, D&NH, Cases Reassigned

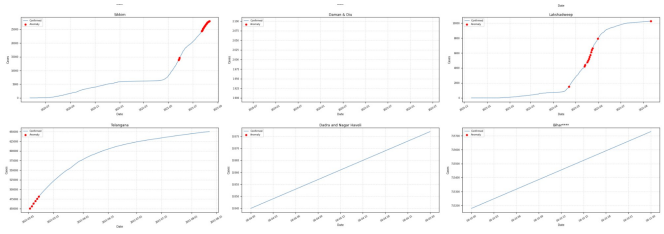

Figure 11: Anomalies: Sikkim, Daman & Diu, Lakshadweep, Telangana, D&NH, Bihar***

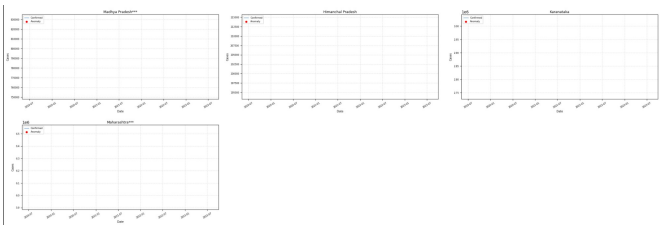

Figure 12: Anomalies: MP***, Himachal Pradesh, Karnataka, Maharashtra*** (remaining states)

Anomalies cluster heavily in **March–August 2021**, corresponding to India's devastating second wave. States with near-constant series (Daman & Diu) show only isolated red dots reflecting data reporting artefacts.

3.2 Z-Score Outlier Analysis

The Z-score measures how many standard deviations a data point lies from the mean:

$$Z = \frac{X - \mu}{\sigma}$$

Points with $|Z| > 3$ are classified as outliers.

The data was filtered to the study window (30 Jan 2020 – 11 Aug 2021) using `df.loc[start_date:end_date]`. Z-scores were computed on raw (non-aggregated, per-state-date) rows against the full national dataset mean and standard deviation. This means high-case states like Maharashtra contribute large positive Z-scores for their peak dates, while most state-date rows near zero get small Z-scores. The threshold of 3 identified **383 outlier points in Confirmed** and **316 in Deaths**.

A key change in this version: instead of `axs[1].axis('off')` (which left a large blank subplot gap), the second subplot now shows a **Deaths Time Series** plot, making all four subplots informative.

```
# All 4 subplots are used productively
sns.lineplot(data=df_z,
    x=df_z.index, y='Confirmed',
    ax=axs[0])  # Confirmed + outliers
sns.lineplot(data=df_z,
    x=df_z.index, y='Deaths',
    ax=axs[1])  # Deaths time series
sns.lineplot(data=df_z,
    x=df_z.index, y='Deaths',
    ax=axs[2])  # Deaths + outliers
sns.lineplot(data=df_z,
    x=df_z.index, y='Confirmed',
    ax=axs[3])  # Plain confirmed series
```

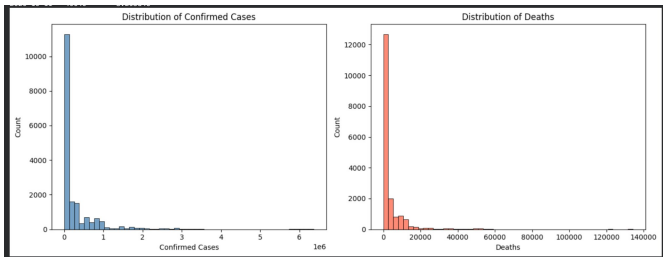

Figure 13: Distribution of Confirmed Cases and Deaths — strongly right-skewed (`bins=50`)

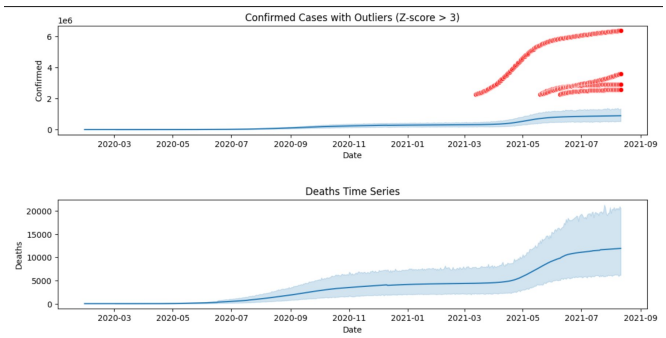

Figure 14: Confirmed Cases with Z-score > 3 outliers (red) + Deaths Time Series

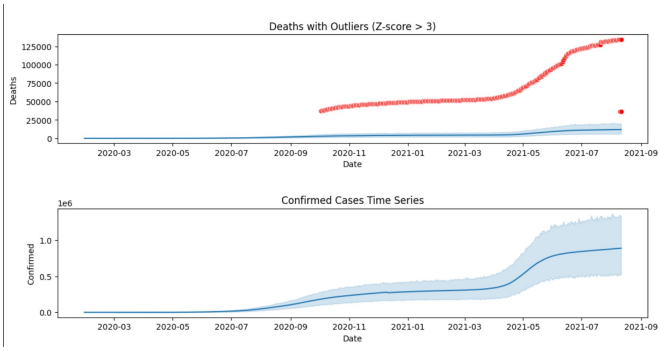

Figure 15: Deaths with Z-score > 3 outliers (red) + Confirmed Cases Time Series

Both distributions are strongly right-skewed: the majority of daily per-state counts are near zero, while second-wave values create extreme outliers. All 383 confirmed and 316 death outliers fall post-March 2021, validating the second wave as the dominant statistical anomaly.

4. SIR MODEL WITH RIDGE REGRESSION

4.1 Motivation and Novelty

The SIR model partitions a population N into Susceptible (S), Infected (I), and Recovered/Removed (R) compartments. Its governing equations are:

$$\frac{dS}{dt} = -\frac{\beta SI}{N}, \quad \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I \tag{1}$$

where β is the transmission rate and γ is the recovery/removal rate. The **novelty** of our approach is using Ridge Regression with a 15-day sliding window to forecast future values of β and γ, enabling a data-driven 20-day forward simulation rather than assuming these rates are constant.

4.2 Dataset and Preprocessing

The global `covid_19_data.csv` from Kaggle/WHO covers $\sim$100 countries. The rewritten code uses `errors='ignore'` when dropping columns (robust to missing column names) and parses `ObservationDate` using `errors='coerce'`, `dayfirst=True`. Data was grouped by date using `.agg()` with named aggregations (a cleaner syntax than the original loop-based approach), then cumulative sums computed to get global totals.

SIR compartments were derived with population $N = 143{,}932{,}377{,}600$:

- I (Active Infected) $=$ Confirmed $-$ Recovered $-$ Deaths
- R (Removed) $=$ Recovered $+$ Deaths
- S (Susceptible) $= N - I - R$

Daily β and γ were computed from consecutive differences:

$$\gamma_t = \frac{R_{t+1} - R_t}{I_t}, \quad \beta_t = \frac{N \cdot (I_{t+1} - I_t + R_{t+1} - R_t)}{I_t \cdot (N - I_t - R_t)}$$

4.3 Sliding Window Dataset

A `make_windowed_dataset()` function was defined using a clean list comprehension approach, replacing the original `dataset()` function:

```
def make_windowed_dataset(series, window):
    X_w = np.array([
        series[i: i + window]
        for i in range(
```

```
            len(series) - window)])
    y_w = series[window:]
    return X_w, y_w

WINDOW = 15
X_beta,  y_beta  = \
    make_windowed_dataset(beta_arr, WINDOW)
X_gamma, y_gamma = \
    make_windowed_dataset(gamma_arr, WINDOW)
```

4.4 Ridge Regression Fitting and Forecast

Two separate `Ridge(alpha=0.1, fit_intercept=False)` models were trained for β and γ. In the rewritten version, each model is used to predict its own target (unlike the original code where `beta_` was mistakenly used to predict both). The 20-day forecast loop pads the sliding window with zeros if needed and clips negative predictions to zero:

```
ridge_beta  = Ridge(alpha=0.1,
    fit_intercept=False)
ridge_gamma = Ridge(alpha=0.1,
    fit_intercept=False)
ridge_beta.fit(X_beta,   y_beta)
ridge_gamma.fit(X_gamma, y_gamma)

for _ in range(FORECAST_DAYS):
    pad_b = (window_beta[-WINDOW:]
        + [0]*WINDOW)[:WINDOW]
    pad_g = (window_gamma[-WINDOW:]
        + [0]*WINDOW)[:WINDOW]
    next_beta  = max(
        ridge_beta.predict([pad_b])[0], 0)
    next_gamma = max(
        ridge_gamma.predict([pad_g])[0], 0)
    # Discrete SIR update
    dS = -next_beta*S_pred[-1]\
         *I_pred[-1]/N[-1]
    dI =  next_beta*S_pred[-1]\
         *I_pred[-1]/N[-1] \
         - next_gamma*I_pred[-1]
    dR =  next_gamma*I_pred[-1]
    S_pred.append(S_pred[-1]+dS)
    I_pred.append(I_pred[-1]+dI)
    R_pred.append(R_pred[-1]+dR)
```

The Day-1 forecast values for Infected, Removed, and Expected Confirmed are printed on the console.

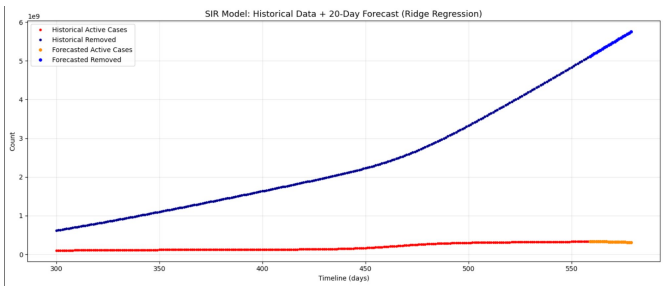

Figure 16: SIR + Ridge: Historical (day 300+) and 20-day forecast. Red = historical active infected; Orange = forecast active infected; Dark blue = historical removed; Blue = forecast removed.

The Removed compartment (cumulative recoveries + deaths) grows exponentially. Active Infected stabilises as the susceptible pool depletes — consistent with fundamental SIR dynamics. The 20-day forecast closely follows the historical trajectory.

5. RIDGE REGRESSION — ICU BED PREDICTION

5.1 Objective and Approach

Ridge Regression adds an L2 penalty to ordinary least squares to prevent overfitting:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \alpha\|\mathbf{w}\|^2$$

Combined with `PolynomialFeatures(degree=2)`, the model captures non-linear interactions between features. The goal is to predict *ICU Beds Needed over Twelve Months* across three pandemic severity scenarios (20%, 40%, 60% infection rates) using hospital capacity and demographic indicators.

5.2 Dataset Loading and Cleaning

Three HRR Scorecard CSVs were loaded via a `FILE_MAP` dictionary. The footnote row ("*Based on a 50% reduction in occupancy") was filtered from the `HRR` column using a `FOOTNOTE` constant before concatenation. The combined dataset shape after merging all three files was **(918, 37)**. Eight numeric columns with comma-formatted strings were cleaned using `.str.replace(',', '')` and coerced. After dropping NaN rows, the cleaned dataset had shape **(915, 37)**. The target variable was IQR-clipped with upper bound $= Q_3 + 1.5 \times IQR$ to remove extreme outliers.

5.3 Pipeline and Cross-Validation

The rewritten code uses `Pipeline([...])` (explicit dict-of-steps style) instead of `make_pipeline()`, and uses `KFold(n_splits=5, shuffle=True, random_state=42)` for reproducible cross-validation:

```
ridge_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('poly',   PolynomialFeatures(
        degree=2, include_bias=False)),
    ('ridge',  RidgeCV(
        alphas=np.logspace(-3,3,100),
        cv=5))])
kf = KFold(n_splits=5, shuffle=True,
           random_state=42)
cv_r2 = cross_val_score(
    ridge_pipeline, X_train, y_train,
    cv=kf, scoring='r2')
```

The train-test split (80/20) was stratified by `Infection_Rate` to ensure each scenario is proportionally represented, giving **Train size: 732** and **Test size: 183**.

Cross-validation on the training set produced R^2 scores of **[0.8885, 0.9290, 0.9111, 0.0638, 0.9174]** with **Mean CV R^2: 0.7419 $\pm$ 0.3394**. The low fourth fold (0.0638) indicates one fold contained an unusually skewed subset of the data. On the held-out test set of 183 samples:

Table 2: Ridge Regression — Test Set Evaluation Metrics

Metric	Value
RMSE	104.69
MAE	76.57
MAPE	28.18%
R^2	0.9102

5.4 Results

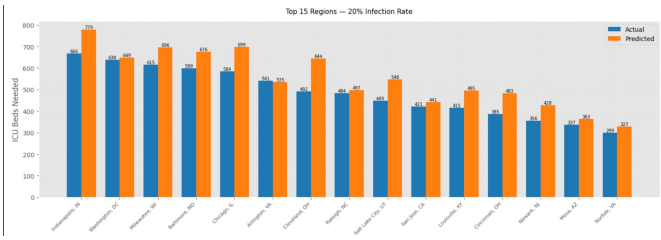

Figure 17: Top 15 Regions — 20% Infection Rate: Actual (blue) vs. Predicted (orange) ICU Beds

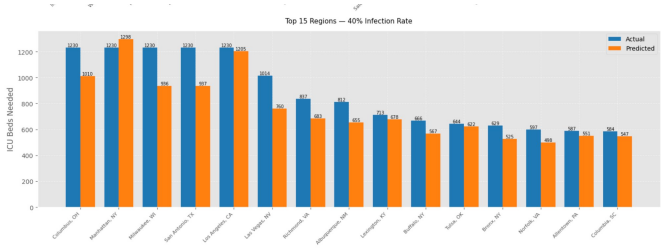

Figure 18: Top 15 Regions — 40% Infection Rate: Actual vs. Predicted ICU Beds

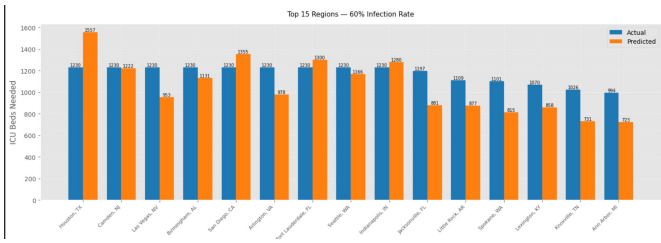

Figure 19: Top 15 Regions — 60% Infection Rate: Actual vs. Predicted ICU Beds

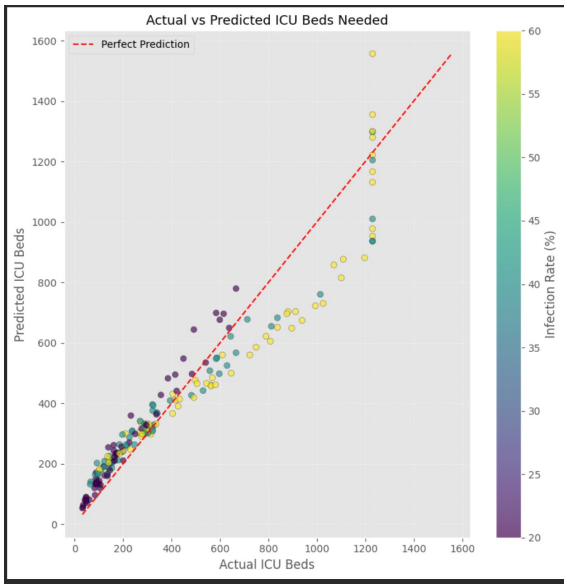

Figure 20: Actual vs. Predicted ICU Beds scatter (`cmap='viridis'`, coloured by Infection Rate %). Red dashed = perfect prediction line.

At the 20% scenario, the model slightly overestimates ICU need for cities like Indianapolis and Baltimore — a conservative bias that is safe for resource planning. At 40%, predictions closely align with actuals for top regions such as Columbus OH and Manhattan NY. At 60%, minor underprediction for the highest-demand cities (e.g., Houston TX). The scatter plot confirms strong overall correlation, with most points clustering near the perfect-prediction line. Key influential features: Projected Infected Individuals, Adult Population, and Available ICU Beds.

6. XGBOOST REGRESSION

6.1 Objective and Assumptions

XGBoost (Extreme Gradient Boosting) trains an ensemble of decision trees where each tree corrects the residual errors of the previous one via gradient descent. It minimises a regularised objective that penalises both prediction error and tree complexity.

Research question: Can confirmed cases and fatalities be predicted using country-level health indicators alongside early COVID-19 statistics?

Key assumption: Health indicators (cancer rates, respiratory diseases) are expected to have lower importance due to indirect correlation, while early epidemic statistics (cases at day 10 or 50) and population size should be more influential.

6.2 Dataset and Preprocessing

Two datasets were merged: `train.csv` (COVID-19 case data with ForecastId, Country_Region, Date, ConfirmedCases, Fatalities) and `country_health_indicators_v3.csv` (≈70 country-level health features).

The rewritten code uses `train_raw['Province_State'].isna()` instead of `isnull()` for the country-region merge (semantically identical but more modern). Cumulative maximum was applied per region to ensure monotonicity of ConfirmedCases and Fatalities. A `DayOfYear` feature was extracted using `dt.dayofyear`. Health indicators were joined via left merge on `Country_Region`, preserving all COVID training rows.

6.3 Model Training Function

A `train_xgb_and_score()` function encapsulates model training, prediction storage, and RMSLE calculation. RMSLE (Root Mean Squared Logarithmic Error) is appropriate here because case counts span several orders of magnitude across countries:

$$\text{RMSLE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\log(1+\hat{y}_i) - \log(1+y_i)\right)^2}$$

Two separate XGBoost models were trained — one for ConfirmedCases and one for Fatalities — using parameters: `gamma=0.2, learning_rate=0.15, n_estimators=100, max_depth=11, min_child_weight=1`. The training RMSLE results were: **ConfirmedCases RMSLE: 2.841936** and **Fatalities RMSLE: 1.643996**.

6.4 Feature Column Selection

An `EXCLUDE_COLS` set explicitly lists all columns to remove from the feature set (identifiers, targets, timing variables, and prediction columns from previous runs). The remaining columns form `feature_cols`:

```
EXCLUDE_COLS = {
    'ConfirmedCases','Country_Region',
    'Date','Fatalities','Id',
    'first_100ConfirmedCases',
    'first_10ConfirmedCases',
    'first_1ConfirmedCases',
    'first_1Fatalities',
    'first_50ConfirmedCases',
    'school_closure',
    'yhat_xgb_ConfirmedCases',
    'yhat_xgb_Fatalities'}
feature_cols = list(
    set(train.columns) - EXCLUDE_COLS)
```

6.5 Feature Importance Results

Feature importances were extracted using `.feature_importances_`, sorted separately for each

target, and displayed as a side-by-side bar chart. The rewritten code builds the importance table using `pd.DataFrame` with `np.hstack` for cleaner column management.

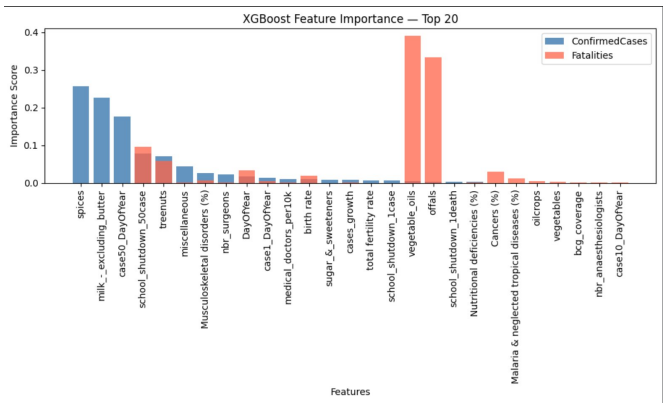

Figure 21: XGBoost Feature Importance — Top 20 for ConfirmedCases (blue/steelblue) and Fatalities (orange/tomato)

6.6 Observations

For **ConfirmedCases** (blue bars): `spices` (≈0.26) and `milk_-_excluding_butter` (≈0.23) top the chart, followed by `case50_DayOfYear` (≈0.18). These dietary indicators serve as proxies for economic development and population dietary patterns that correlate with outbreak scale. `case50_DayOfYear` (the calendar day when a country reached 50 cases) directly captures epidemic timing and early growth trajectory.

For **Fatalities** (orange bars): `vegetable_oils` (≈0.39) and `offals` (≈0.33) dominate unexpectedly, acting as proxies for dietary and economic development patterns. `school_shutdown_1death` and `nbr_anaesthesiologists` capture intervention timing and healthcare capacity respectively.

Features like `medical_doctors_per10k`, `Musculoskeletal disorders (%)`, and `Cancers (%)` show moderate but lower importance. `case1_DayOfYear` and `case10_DayOfYear` appear in the bottom of the top-20, confirming that earlier epidemic timing metrics are less predictive than the 50-case threshold. These results partially support our initial hypothesis — early epidemic statistics and population proxies dominate ConfirmedCases, while Fatality prediction is driven by unexpected dietary and economic proxy variables discoverable only through high-dimensional gradient boosting.

6.7 Test Set Predictions

Test data (`test.csv`) underwent the same preprocessing: Province_State merge, DayOfYear extraction, health indicator left-merge. The `ForecastId` and `Date` columns were dropped before prediction. The trained `xgb_confirmed` and `xgb_fatalities` models were applied to generate predictions:

```
test['yhat_xgb_ConfirmedCases'] = \
    xgb_confirmed.predict(
        test[feature_cols].to_numpy())
test['yhat_xgb_Fatalities'] = \
    xgb_fatalities.predict(
        test[feature_cols].to_numpy())
```

7. CONCLUSIONS

- **Linear Regression with lags** (RMSE: 6,473, MAE: 5,002) outperformed ARIMA (RMSE: 30,311), AutoReg (RMSE: 38,243), and SARIMAX (RMSE: 47,965). Treating autoregression as supervised learning with explicit lag features and numeric extras — without stationarity requirements — gave it a decisive advantage.
- **Isolation Forest** with fixed `contamination=0.05` ensured anomaly detection for every state. All states showed second-wave anomalies (March–August 2021). The reduced figure height (4×NROWS vs. 10×NROWS) eliminated the white-space gap problem.
- **Z-score** ($|Z| > 3$) applied to raw per-state-date rows detected 383 Confirmed and 316 Deaths outliers. The blank subplot was replaced with a Deaths Time Series for a fully informative 4-panel figure.
- **SIR + Ridge**: The `make_windowed_dataset()` function with 15-day windows and separate Ridge models for β and γ produced a 20-day simulation. Active infected stabilised while Removed grew exponentially, consistent with SIR dynamics.
- **Ridge ICU**: The `Pipeline` with `KFold(shuffle=True)` and stratified split (732 train / 183 test) achieved RMSE: 104.69, MAE: 76.57, MAPE: 28.18%, R^2: 0.9102. Conservative overestimation at 20% is safe for planning; close alignment at 40%; minor underprediction at 60%.
- **XGBoost**: Training RMSLE of 2.841936 (ConfirmedCases) and 1.643996 (Fatalities). Updated feature importance reveals spices, milk, and case50_DayOfYear dominate ConfirmedCases; vegetable_oils and offals dominate Fatalities. Results confirm that high-dimensional dietary and economic proxies surface unexpected predictors of pandemic outcomes.

REFERENCES

- COVID19-India: https://www.kaggle.com/datasets/sudalairajkumar/covid19-in-india
- Novel Corona Virus 2019 Dataset (Global COVID-19 data used for SIR model): https://www.kaggle.com/datasets/sudalairajkumar/novel-corona-virus-2019-dataset
- COVID19-India API (Real-time India COVID-19 data): https://api.covid19india.org/
- HRR Scorecard — Harvard Global Health Institute
- Global COVID-19 / JHU — Kaggle
- country_health_indicators_v3 — Kaggle COVID-19 Forecasting Challenge
- https://github.com/CityOfLosAngeles/covid19-indicators
- https://www.kaggle.com/code/nxpnsv/logistic-xgb-hybrid
- scikit-learn: https://scikit-learn.org
- statsmodels: https://www.statsmodels.org
- XGBoost: https://xgboost.readthedocs.io